

결정 트리에서 랜덤 포레스트까지 — 질문의 나무, 다수결의 숲

스무고개처럼 질문을 쌓아 가르는 결정 트리, 그리고 흔들리는
한 그루를 수백 그루의 다수결로 다잡는 랜덤 포레스트

결정 트리는 "예/아니오" 질문으로 데이터를 거듭 쪼개 **순수한** 무리를 만듭니다. 어떤 질문이 좋은지는 **불순도(지니·엔트로피)**로 따지고, 규칙만으로 답하니 사람이 그대로 읽을 수 있죠. 다만 한 그루는 쉽게 흔들리고 외웁니다. 그래서 서로 다르게 기른 **수백 그루를 다수결**로 묶은 게 랜덤 포레스트예요 — 두 번의 무작위(행·열)가 트리들을 닮지 않게 만들어 안정성과 정확도를 함께 끌어올립니다.

오늘 끝내면 할 수 있는 것

네 가지면 충분합니다.

01

트리를 읽고 만든다

루트·내부 노드·잎·깊이로 트리를 해부하고, **분할변수 j·분할점 s**를 모든 후보에서 시험해 불순도(지니·정보 이득)를 가장 줄이는 질문을 고르는 과정을 설명한다.

02

과적합을 다스린다

트리가 **왜 쉽게 외우는지** 알고, **깊이 제한·잎의 최소 인원** 같은 **가지치기**로 단순하게 만드는 원리를 안다.

03

왜 숲인지 안다

앙상블이 한 모델보다 나은 두 조건(독립·0.5보다 나음)과, **Bagging(행)·Aggregating(종합)·Random subspace(열)** 무작위가 다양성을 만드는 원리를 그림으로 푼다.

04

숲을 읽는다

OOB로 공짜 검증을 하고 **변수 중요도(섞기)**로 영향력을 보며, **그루 수·분기당 변수 수**가 무엇을 조절하는지 안다.

직선 말고, 질문으로 가른다면

로지스틱 회귀는 직선 하나를 그어 확률로 가릅니다. 그런데 사람은 보통 **질문을 거듭해** 판단합니다 — 결정 트리가 바로 그 방식이에요.

우리는 이미 트리로 생각합니다. "비 와? → 그럼 우산. 안 와? → 더워? → 그럼 반팔". 조건을 하나씩 물어 답에 닿는 이 **스무고개**가, 그대로 분류 모델이 됩니다.

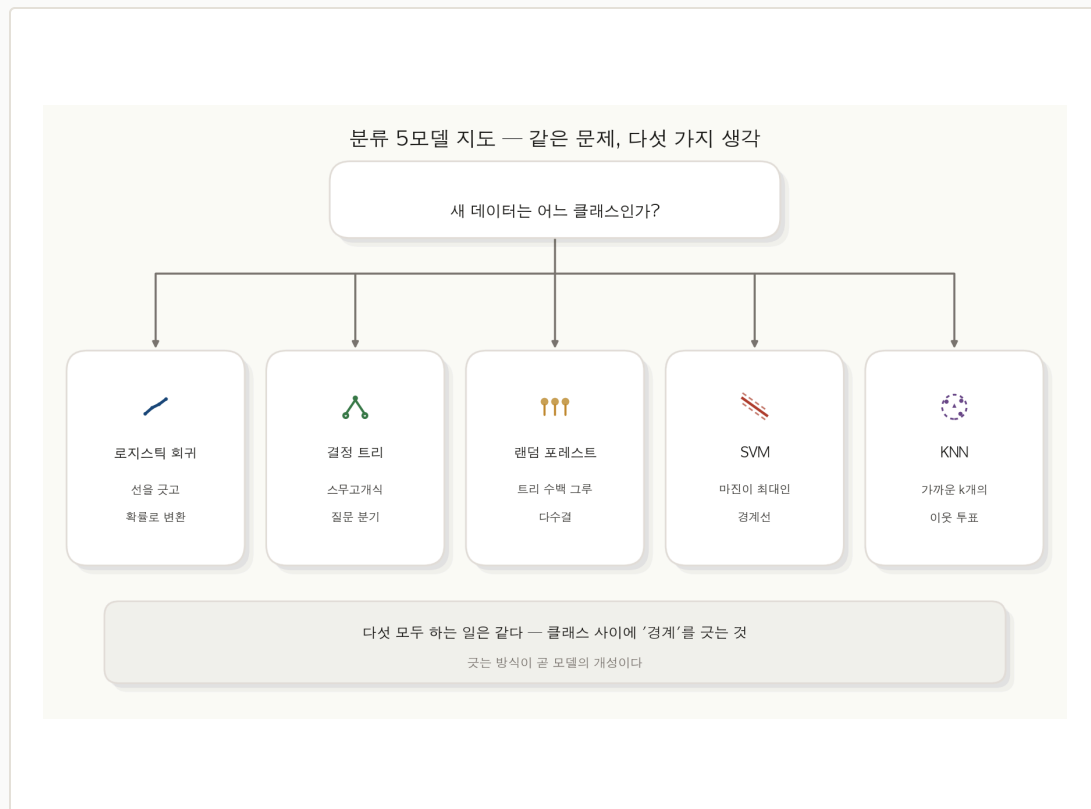
트리는 데이터를 '나뉘' 갑니다. 로지스틱이 경계선을 **그었다면**, 트리는 "공부 2시간 이하인가?" 같은 질문으로 데이터를 **둘로 쪼개고**, 각 조각을 또 쪼개며 점점 **비슷한 것끼리** 모읍니다.

그래서 **출력이 직선도 확률도 아닙니다**. 트리의 답은 **규칙의 연쇄** — "공부 2시간 이하 & 출석 70% 이하 → 불합격"처럼 사람 말로 그대로 옮길 수 있어요. 이 **투명함**이 트리의 가장 큰 매력입니다.

오늘의 큰 그림. 먼저 트리 한 그루를 깊이 보고 (§1~§3), 그 한 그루가 흔들린다는 약점에서 출발해 **수백 그루의 숲**(랜덤 포레스트, §4~§5)으로 확장합니다 — 트리에서 숲으로 가는 한 흐름이에요.

다섯 모델 중 둘 — 트리와 숲의 자리

분류 모델들이 답하는 질문은 하나예요 — "새 데이터는 어느 클래스인가?". 로지스틱이 선을 그었다면, 결정 트리는 **질문의 계단**을 쌓고, 랜덤 포레스트는 그 **트리를 수백 그루 모읍니다**. 이 모듈에서 트리와 숲을 함께 봅니다.



01 로지스틱 회귀

직선을 긋고 그 값을 **확률로 변환**합니다. 부드러운 경계 하나로 가르는 방식이죠.

02 결정 트리 — 먼저 다룰 모델

스무고개식 질문으로 데이터를 거둬 쪼갭니다. 경계가 부드러운 선이 아니라 **축에 평행한 계단**이에요.

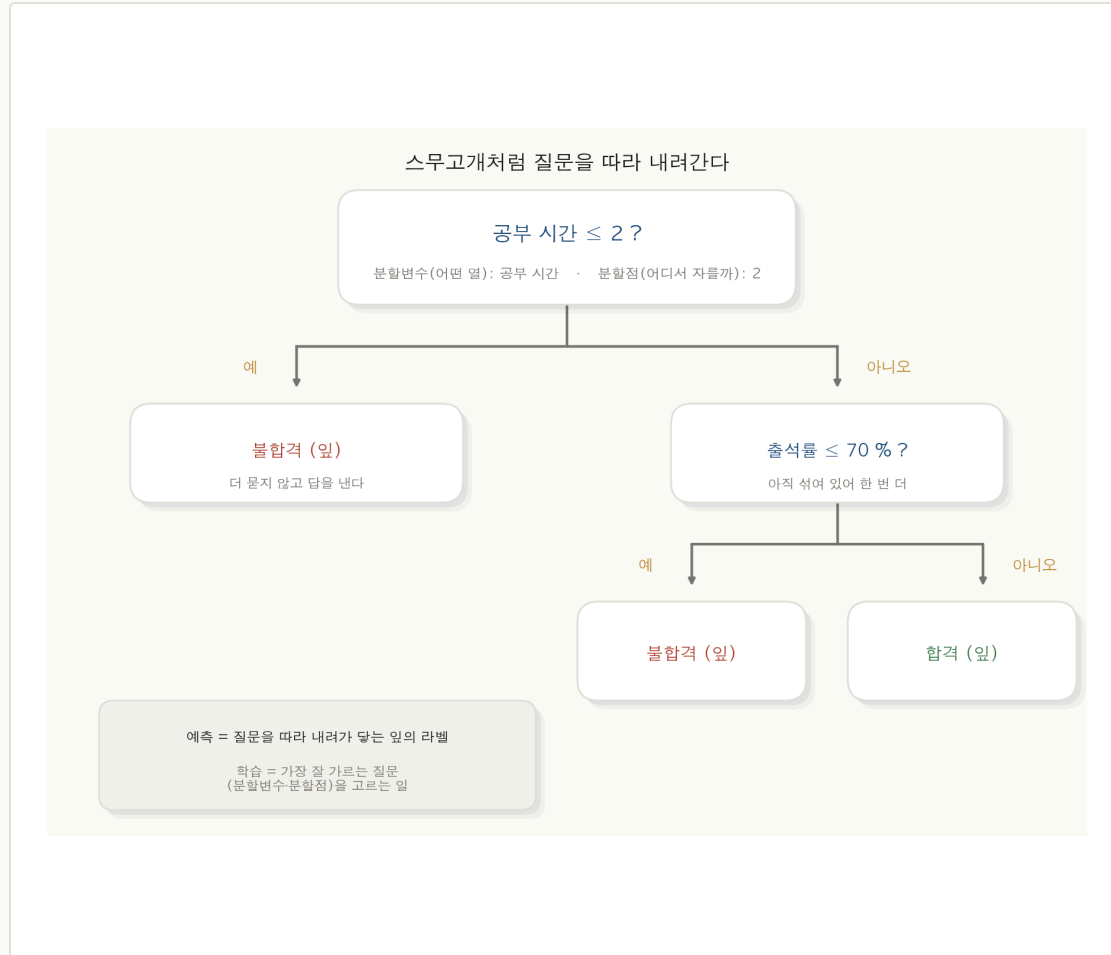
03 랜덤 포레스트 — 이어서 다룰 모델

트리 한 그루는 불안정합니다. 그래서 **수백 그루의 다수결**로 묶죠 — 트리가 그 출발점입니다.

04 SVM·KNN

최대 여백 경계를 찾거나(SVM), 가까운 이웃 k개의 투표에 맡깁니다(KNN). 같은 문제, 또 다른 발상이죠.

합격을 가르는 스무고개 — 질문을 따라 내려간다



위에서 질문에 답하며 내려가면 끝에 예측이 있다 — 학습은 '가장 잘 가르는 질문'을 고르는 일.

01 위에서부터 질문을 던진다

맨 위 질문은 "공부 시간 ≤ 2 ?". **예**면 왼쪽, **아니오**면 오른쪽으로 갈라집니다. 한 번의 질문이 데이터를 **두 무리**로 가르는 거죠.

02 답이 날 때까지 또 묻는다

오른쪽 무리는 아직 섞여 있어 한 번 더 묻습니다 — "출석률 $\leq 70\%$ ". 이렇게 **충분히 순수해질 때까지** 질문을 이어가요.

03 끝에 도착하면 그게 예측

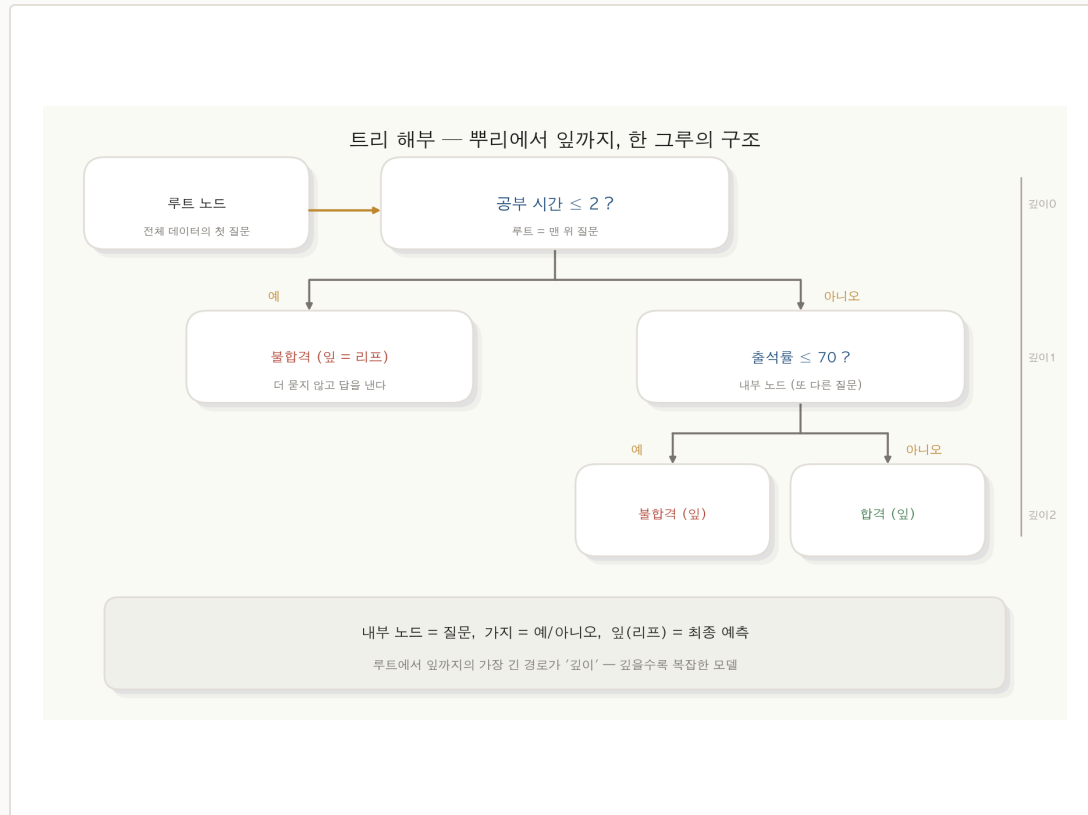
더 안 묻는 끝 칸이 **잎(리프)**입니다. 새 학생이 오면 질문을 따라 내려가다 닿는 잎의 라벨 — "합격"이 곧 예측이에요.

04 학습 = 좋은 질문 고르기

이 트리를 **사람이 만든 게 아닙니다**. 어떤 변수로, 어디서 쪼개야 가장 잘 갈리는지를 데이터에서 찾아내는 것 — 그게 §2에서 배울 학습이에요.

트리의 구조 — 루트에서 잎까지

트리는 **노드**(질문 상자)와 **가지**(예/아니오 갈래)로 이뤄집니다. 맨 위가 **루트**, 또 가지를 치면 **내부 노드**, 더 안 묻고 답을 내면 **잎(리프)**이에요. 루트에서 잎까지의 가장 긴 경로가 **깊이** — 깊을수록 복잡한 모델입니다.



01 루트 노드 — 첫 질문

전체 데이터가 모여 처음 갈라지는 자리입니다. 가장 잘 가르는 질문이 여기 놓여요 — 트리의 출발점이지요.

02 내부 노드 — 또 묻는 곳

아직 섞여 있어 **한 번 더 쪼개는** 질문 상자입니다. 루트도 내부 노드의 하나 — 다만 맨 위라는 점만 달라요.

03 잎(리프) — 최종 예측

더 안 묻고 **답을 내는** 끝 칸입니다. 그 안에 모인 데이터의 **다수 클래스**가 예측이 돼요 (회귀라면 평균).

04 깊이 — 질문을 몇 번 했나

루트에서 잎까지 거친 질문 수입니다. 깊이를 키우면 더 잘게 가르지만 — **과적합**의 위험도 함께 자라요(§3).

어디서 쪼갤까 — 모든 후보를 일일이 시험한다



분할변수 j (어떤 열로)와 분할점 s (어느 값에서)의 모든 조합을 따져, 불순도가 가장 낮은 곳을 고른다.

01 질문 = 변수 + 분할점

트리의 한 질문은 " $x_1 \leq 2$?"처럼 어떤 변수(j)를 어떤 값(s)에서 자르냐로 정해집니다. 둘을 정하면 데이터가 둘로 갈라져요.

02 후보를 전부 줄 세운다

변수가 x_1, x_2 면 — x_1 은 1·2·3·4에서, x_2 는 50·60·70·80에서 자를 수 있습니다. 가능한 (j, s) 조합을 하나도 빠지 않고 늘어놓아요.

03 후보마다 쪼개 본다

각 후보로 실제로 데이터를 둘로 나눠 보고, 그렇게 갈린 두 무리가 얼마나 순수한지를 쟁니다 — 그 자가 다음 장의 불순도예요.

04 불순도가 가장 낮은 곳 선택

여기선 $x_1 \leq 2$ 가 가장 깨끗하게 갈랐습니다. 그래서 이 질문이 채택돼요 — 트리는 이 완전 탐색을 노드마다 반복합니다.

'얼마나 섞였나'를 숫자로 — 불순도

불순도(impurity)는 한 노드 안에 클래스가 **얼마나 뒤섞여 있는지**를 재는 점수입니다. 한 클래스만 모이면 **0**(가장 순수), 반반이면 가장 큼니다. 트리의 목표는 쪼갤 때마다 이 불순도를 **최대한 줄이는** 거예요.

불순도 — '얼마나 섞였나'를 숫자로

순도 높은 노드

9 : 1



$$G = 1 - 0.9^2 - 0.1^2 = 0.18$$

거의 한 클래스 — 좋은 분할

반반 노드

5 : 5



$$G = 1 - 0.5^2 - 0.5^2 = 0.50$$

최악으로 섞임 — 나쁜 분할

지니 불순도 = 노드에서 둘을 뽑았을 때 서로 다른 클래스일 확률 — 작을수록 순수하다

01 순수 = 한 클래스만

앞이 전부 "합격"이면 망설일 게 없죠. 한쪽 클래스로 **쓸릴수록** 불순도가 0에 가까워집니다.

02 최악 = 반반

합격·불합격이 **5:5**면 동전 던지기와 같습니다. 가장 섞인 상태 — 불순도가 최대예요.

03 9:1은 꽤 순수하다

열 중 아홉이 한 클래스면 거의 정해진 거죠. 5:5보다 **훨씬 낮은** 불순도가 나옵니다.

04 쪼개는 이유가 여기 있다

좋은 질문이란 **섞인 무리를 순수한 무리로** 가르는 질문입니다. 그 '얼마나 순수해졌나'를 재려고 불순도가 필요해요.

둘을 뽑아 다를 확률 — 지니 계수

불순도를 재는 **대표적인** 자입니다. 식은 $Gini = 1 - \sum_k p_k^2$ — 노드에서 **아무 둘을 뽑았을 때 서로 다른 클래스일 확률**로 읽으면 직관이 살아요. 값은 **0(순수) ~ 0.5(반반)** 사이입니다.

0

한 클래스만 — 가장 순수

전부 한 클래스($p_k = 1$)면 $\sum_k p_k^2 = 1$ 이라 $Gini = 1 - 1 = 0$. 둘을 뽑아도 늘 같은 클래스 — 더 쪼갤 이유가 없죠.

0.18

[9,1] — 꽤 순수

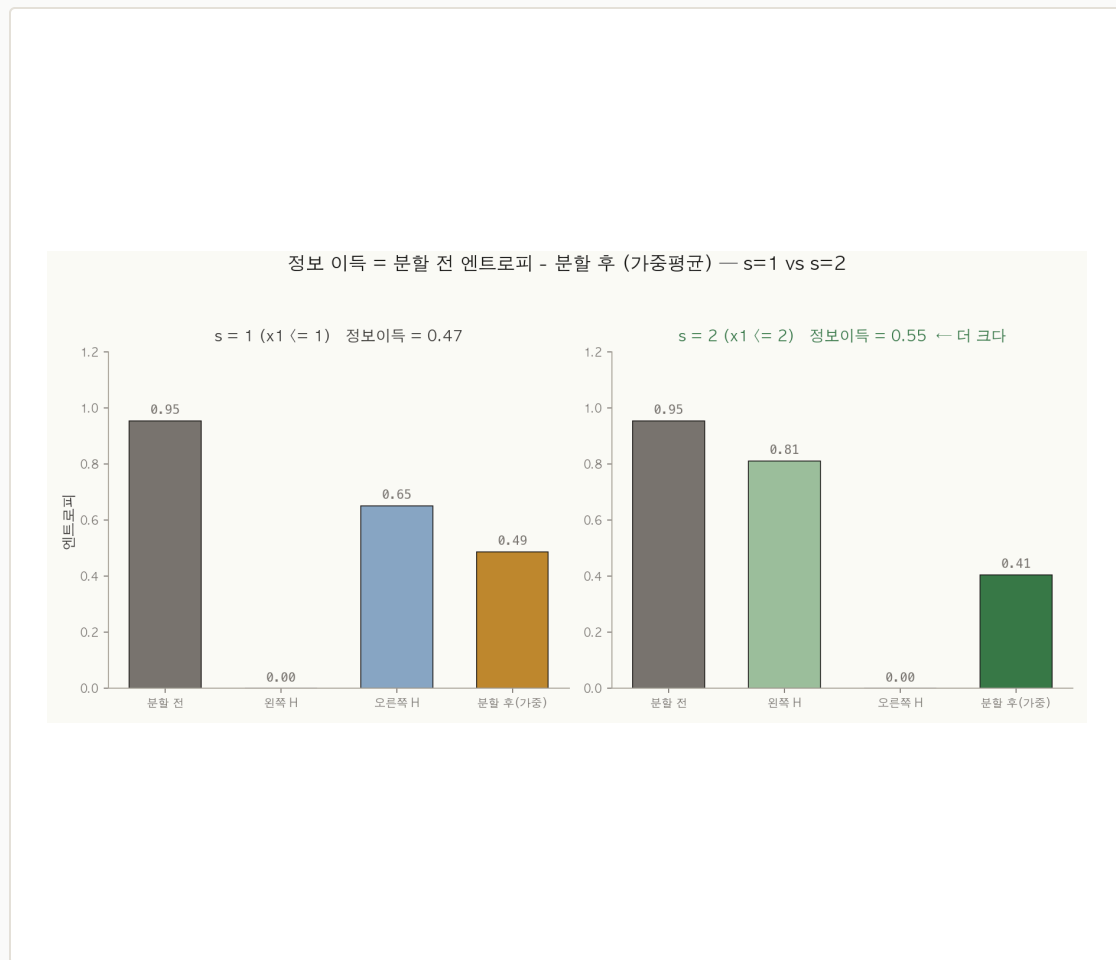
비율이 0.9, 0.1이면 $1 - (0.9^2 + 0.1^2) = 0.18$. 거의 한 클래스라 **낮은 불순도** — 좋은 분할의 결과예요.

0.50

[5,5] — 반반(최댓값)

반반이면 $1 - (0.5^2 + 0.5^2) = 0.50$. 이진 분류에서 지니의 **최댓값** — 가장 섞여 더 쪼갤 이유가 큰 상태입니다.

분할 전후의 불확실성 차이 — 정보 이득



정보 이득 = (분할 전 엔트로피) - (분할 후 가중평균 엔트로피) — s=2가 s=1보다 더 크게 줄인다.

01 엔트로피 — 또 다른 자

지니 말고 **엔트로피**로도 섞임을 잹니다. $H = -\sum_k p_k \log_2 p_k$ — 반반이면 1, 한 클래스면 0. 지니와 **재는 대상은 같고** 계산식만 달라요(보통 결과도 비슷).

02 정보 이득 = 줄어든 불확실성

$IG = H(\text{부모}) - \sum_c \frac{n_c}{n} H(c)$ — 쪼개기 **전** 엔트로피에서, 쪼갠 **후** 두 무리의 엔트로피를 **크기**(n_c/n)로 **가중평균**해 뺀 값입니다. **얼마나 깨끗해졌나**가 곧 정보 이득이에요.

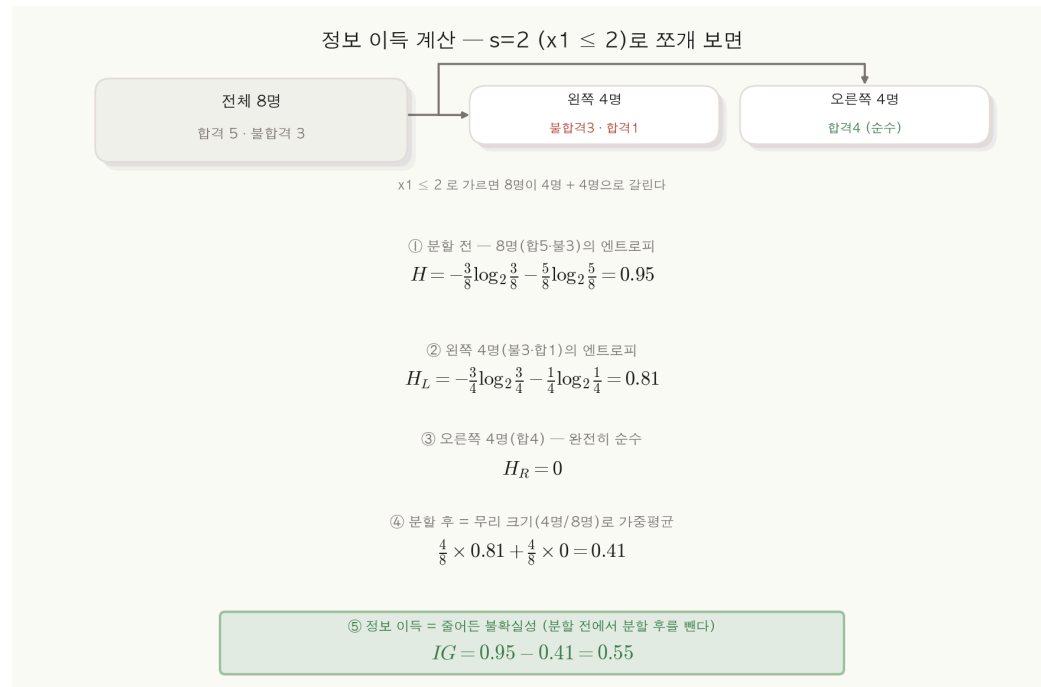
03 분할점에 따라 이득이 다르다

같은 변수라도 자르는 위치에 따라 이득이 달라요. s=1은 한쪽만 깔끔히 갈라 이득 **0.47**, s=2는 한 무리를 완전히 순수하게 만들어 이득 **0.55**(분할 전 엔트로피 0.95 기준).

04 정보 이득이 큰 쪽을 고른다

s=2가 더 **크게 불확실성을 줄였으니** 그 분할이 채택됩니다 — §2.1의 "불순도 최소"와 "정보 이득 최대"는 **같은 말의 앞뒤**예요.

숫자로 따라가 보기 — 정보 이득은 이렇게 나온다



01 ① 분할 전을 먼저 잔다

쪼개기 전 8명은 합격5·불합격3으로 섞여 있어요. 그 엔트로피가 **0.95** — 1(반반)에 가까운, 꽤 불확실한 상태입니다.

02 ② 두 쪽을 각각 잔다

$x_1 \leq 2$ 로 가르면 8명이 **왼쪽 4명·오른쪽 4명**으로 갈려요. 왼쪽은 불합격3·합격1(엔트로피 **0.81**), 오른쪽은 합격4뿐이라 **완전히 순수(0)**입니다.

03 ③ 크기로 가중평균

두 무리가 각 **4명 / 전체 8명**이니 $\frac{4}{8}, \frac{4}{8}$ 로 가중평균 — $0.5 \times 0.81 + 0.5 \times 0 = 0.41$. **분할 후**의 평균 불확실성이예요.

04 ④ 줄어든 만큼이 이득

$IG = 0.95 - 0.41 = 0.55$. 0.95만큼 헛갈리던 게 0.41로 줄었으니, **0.55만큼 깨끗해진 것**. 트리는 이 값이 가장 큰 분할을 고릅니다.

학생 8명(합격5·불합격3)을 $x_1 \leq 2$ 로 쪼개는 한 번의 계산 — 엔트로피 두 번, 가중평균 한 번, 빼기 한 번.

끝까지 자라면 외운다 — 멈추는 법, 가지치기

트리는 불순도가 0이 될 때까지 쪼갤 수 있습니다 — 얼마다 한 명씩 가둘 정도로요. 하지만 그건 데이터를 외운 것(과적합)이지 배운 게 아니에요. 그래서 일부러 멈추거나 잘라내는 가지치기가 필요합니다.

01 끝까지 자란 트리 = 암기

얼마다 데이터 한 개씩 들어갈 때까지 쪼개면 train 정확도는 100% — 하지만 노이즈까지 외워 새 데이터에 무너져요.

02 max_depth — 깊이 상한

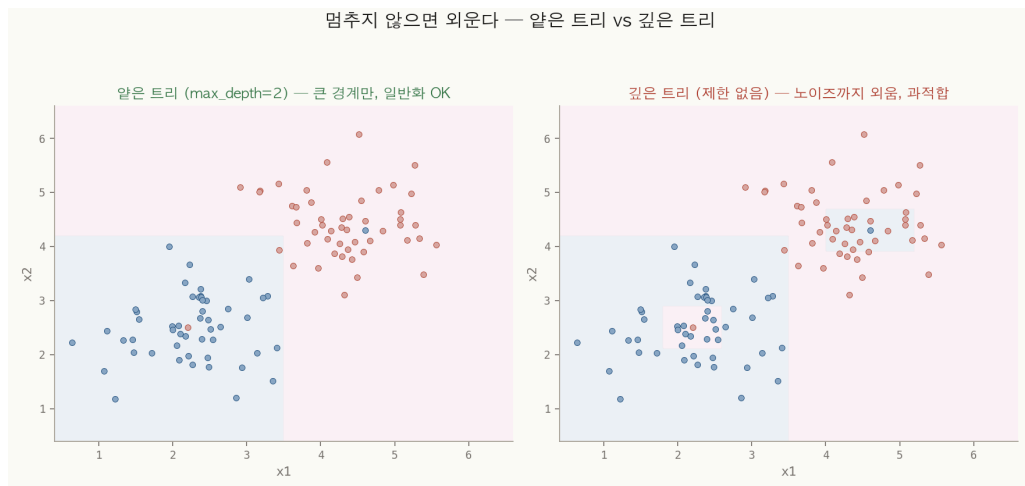
질문을 몇 번까지만 하도록 깊이에 천장을 겁니다. 알수록 단순하고 일반화가 잘 되죠 — 가장 많이 쓰는 손잡이예요.

03 min_samples — 최소 인원

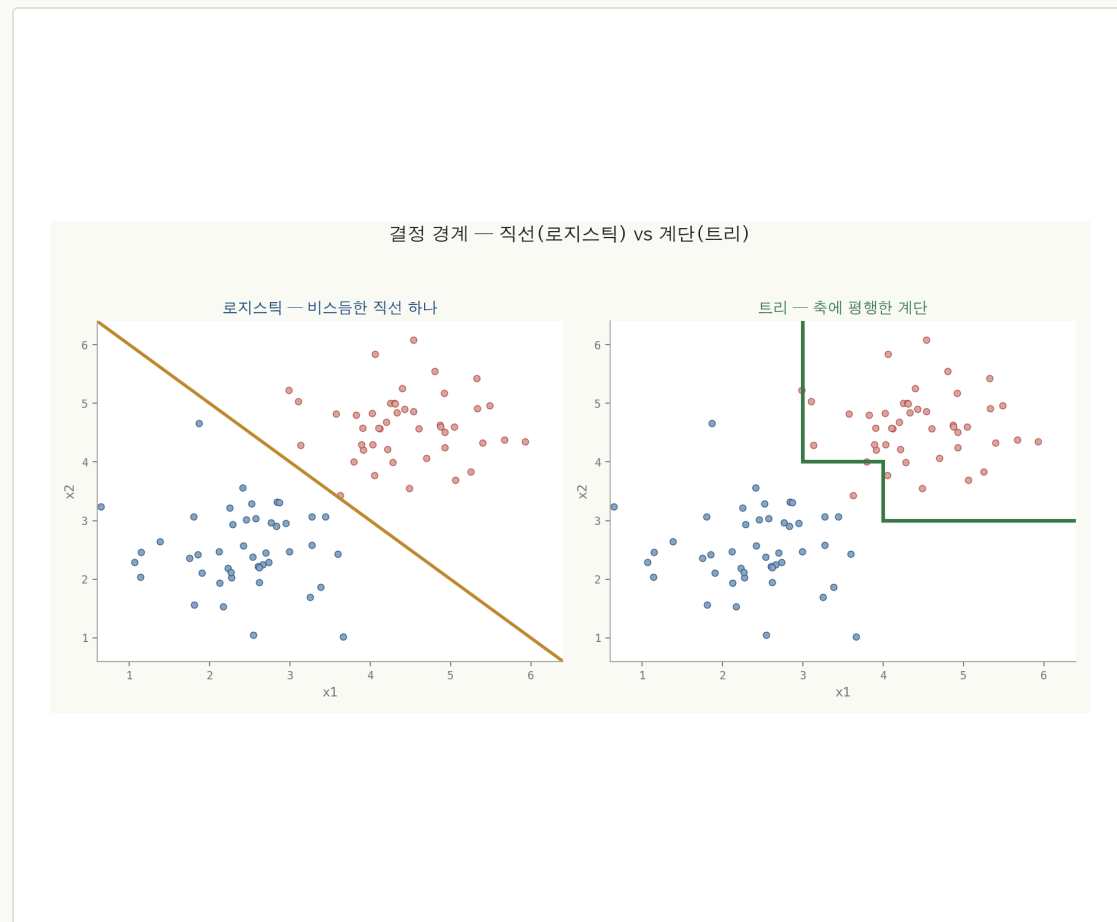
잎(또는 분할)에 최소 몇 개는 있어야 한다는 규칙입니다. 한두 개짜리 잎을 막아 자잘한 암기를 차단해요.

04 사전 가지치기 vs 사후 가지치기

자라기 전에 멈추거나(pre), 다 키운 뒤 쓸모없는 가지를 쳐냅니다(post). 둘 다 목표는 같아요 — 꼭 필요한 만큼만 복잡하게.



트리가 그리는 경계는 왜 계단 모양일까



질문이 '한 변수 $\leq$ 값' 꼴이라 트리 경계는 늘 축에 평행한 계단 — 로지스틱은 확률은 S자(비선형)지만 결정 경계는 직선이라, 비스듬한 선 하나로 가른다.

01 질문 하나 = 축에 평행한 칼금

" $x_1 \leq 2$?"는 x_1 축에 수직인 선으로 평면을 가릅니다. 한 번에 한 변수만 보니 경계가 늘 가로 아니면 세로예요.

02 쪼갤수록 계단이 쌓인다

질문을 거듭하면 이 칼금들이 차곡차곡 포개져 계단 모양 경계가 됩니다. 비스듬한 직선 하나로 끝낸 로지스틱과는 그림부터 달라요.

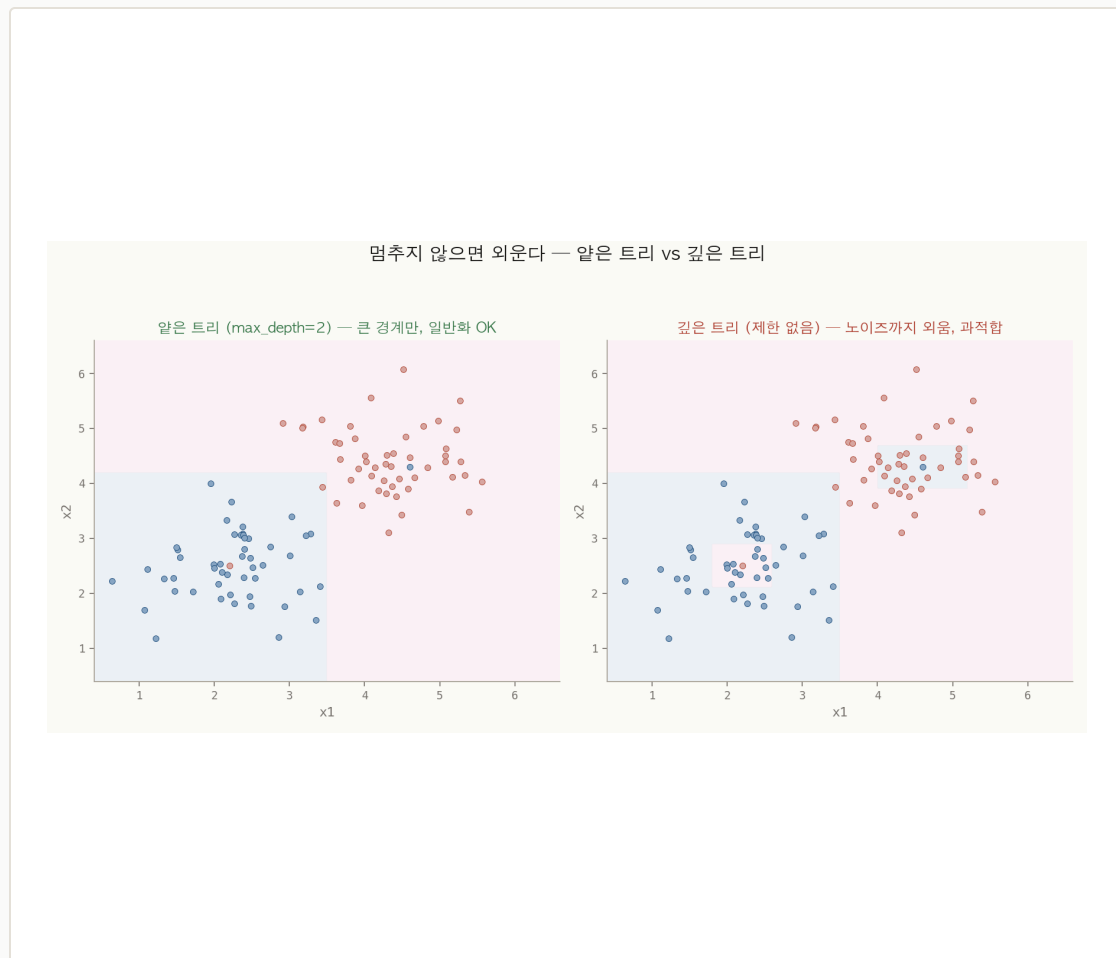
03 비선형 경계도 흉내 낸다

계단을 잘게 쪼개면 곡선 같은 경계도 근사할 수 있습니다. 로지스틱은 확률은 S자라도 결정 경계는 직선 하나뿐 — 트리는 그 한계를 넘는 표현력이죠.

04 대신 비스듬한 경계엔 약하다

데이터가 대각선으로 갈린다면, 트리는 계단을 여러 번 꺾어야 합니다. 로지스틱이라면 선 하나로 끝날 일어요 — 모델마다 잘하는 모양이 달라요.

얕은 트리 vs 깊은 트리 — 어느 쪽이 더 나을까



같은 데이터, 깊이만 다르다 — train 점수 1등은 오른쪽이지만 새 데이터 점수 1등은 왼쪽이다.

01 얕은 트리 — 큰 줄기만

깊이를 2로 묶으면 **굵직한 경계 두어 개**로 끝납니다. 자잘한 예외는 놓치지만, 큰 패턴을 잡아 **새 데이터에도 통하는** 모델이에요.

02 깊은 트리 — 노이즈까지

깊이 제한을 풀면 **이상치 한 점**까지 감싸려고 자잘한 영역을 만듭니다. train에선 거의 다 맞하지만 — 그 영역들이 곧 **암기의 흔적**이에요.

03 들통은 test에서만 난다

과적합한 깊은 트리는 train 점수가 높게 나옵니다. 진짜 실력은 **따로 떼어둔 test**에서 드러나죠 — 회귀에서 본 train/test 구분이 그대로 적용됩니다.

04 그래서 깊이를 조준한다

너무 얕으면 과소적합, 너무 깊으면 과적합. **깊이를 몇 가지 값으로 바꿔 가며** 새 데이터 (test) 점수가 가장 높은 지점을 고릅니다.

결정 트리의 장점과 약점 — 왜 숲으로 가나

여기서부터 **한 그루(결정 트리)** → **숲(랜덤 포레스트)**으로 넘어갑니다. 트리는 장점이 분명한 만큼 약점도 분명한데, 바로 그 **약점**이 숲으로 가는 이유예요.

왜 트리를 쓰는가

장점 Strengths

전처리가 거의 필요 없다

질문이 **값의 크기 순서**만 보기에, 스케일을 맞추거나(정규화) 결측을 메우는 손질이 로지스틱만큼 까다롭지 않아요.

스케일·단위에 무관

"공부 시간 ≤ 2 "는 시간을 분으로 바꿔도 자르는 **위치**만 옮길 뿐 결과는 같습니다. 변수 단위가 제각각이어도 끔찍없어요.

사람이 그대로 읽는다

예측의 근거가 **규칙의 연쇄**라 "왜 불합격인지"를 말로 설명할 수 있습니다 — 로지스틱의 계수와는 또 다른 종류의 투명함이죠.

왜 흔들리는가

약점 Weaknesses

불안정하다

데이터가 조금만 바뀌어도 **루트의 질문부터** 통째로 달라질 수 있습니다. 위쪽 분할이 흔들리면 그 아래가 전부 따라 흔들려요.

노이즈에 민감하다

이상치 몇 개를 **감싸려고** 자잘한 가치를 뺏습니다 — 적은 노이즈에도 경계가 들쭉날쭉해지죠.

쉽게 과적합한다

멈추지 않으면 데이터를 통째로 외웁니다. **그래서 숲이 필요해요** — 트리 수백 그루의 다수결로 흔들림을 평균 내는 게 지금부터 볼 랜덤 포레스트입니다.

한 명의 전문가보다, 여러 명의 다수결

퀴즈쇼의 "청중 찬스"처럼, 한 사람은 틀려도 **여러 사람의 다수결**은 자주 맞습니다. 이 발상을 모델에 옮긴 것이 앙상블이예요.

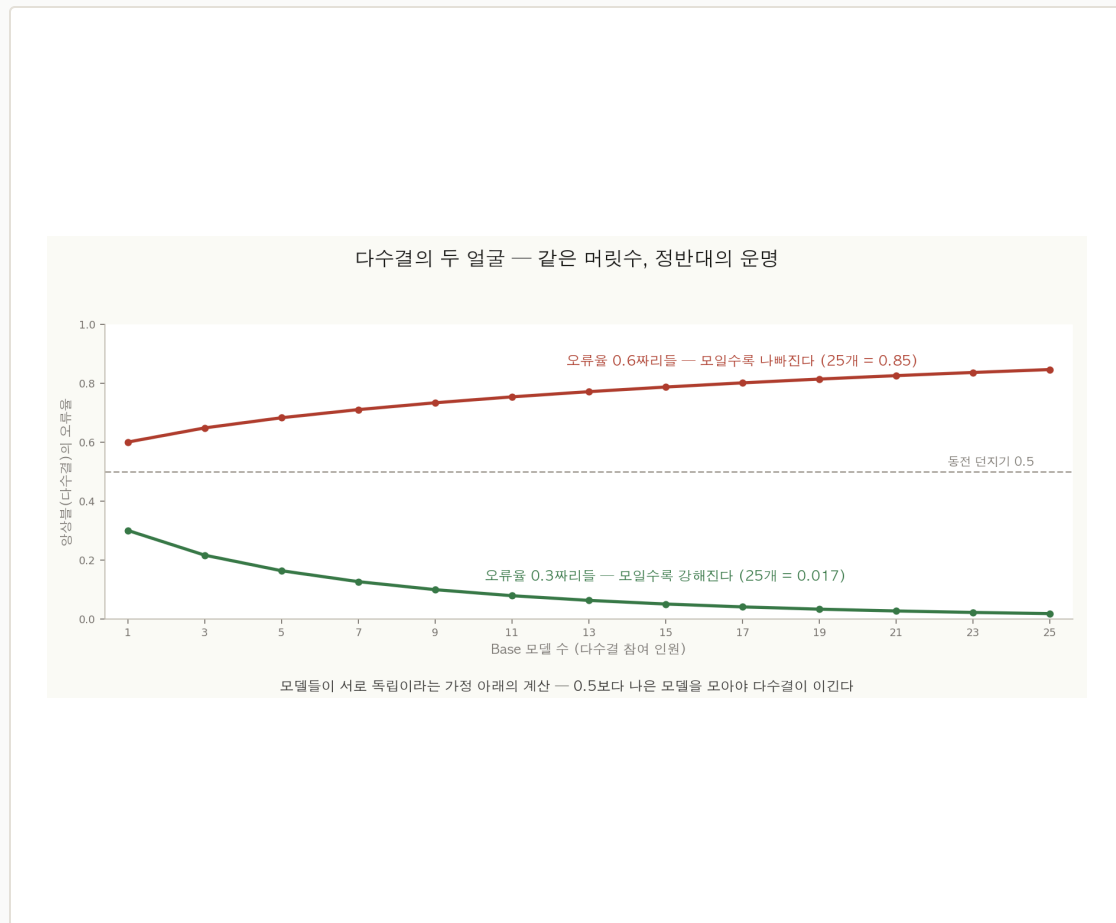
앙상블(Ensemble)은 여러 모델의 예측을 모읍니다. 여러 **Base 모델**의 예측을 분류면 **다수결**로, 회귀면 **평균**으로 합쳐 하나의 답을 냅니다. 개별 모델이 약해도 모으면 더 나은 예측이 됩니다.

핵심은 "**실수가 겹치지 않는다**"입니다. 모델들이 **서로 다른 실수**를 한다면, 한 모델이 틀린 자리를 다른 모델들이 메워줍니다. 다수결을 거치면 개별 실수들이 **서로 상쇄**되며 지워지죠.

그래서 **다양성이 생명**입니다. 똑같은 모델 수백 개는 똑같이 틀립니다 — 모아도 한 개와 다를 게 없어요. 모델들이 **서로 닳지 않아야** 비로소 모은 보람이 생깁니다. 이 다양성을 어떻게 만드느냐가 §4 후반의 주제예요.

다만 공짜는 아닙니다. 아무 모델이나 많이 모은다고 좋아지진 않아요. 다수결이 이득이 되려면 **두 가지 조건**이 맞아야 합니다 — 모델들이 서로 독립이고, 동전 던지기보다는 나을 것. 다음 장에서 수치로 확인합니다.

모을수록 강해지려면 — 두 조건이 맞아야 한다



독립인 모델들의 다수결 오류율 — 0.5보다 나으면(초록) 모일수록 0에 수렴하지만, 0.5보다 못하면(빨강) 모일수록 1로 치솟는다.

01 조건 ① — 서로 독립

모델들이 **서로 다른 실수**를 해야 합니다. 다 같이 똑같이 틀리면 다수결로 모아도 그 실수가 그대로 남아요. 독립이라야 실수들이 서로 상쇄됩니다.

02 조건 ② — 동전보다는 나은

각 모델의 정확도가 **반반(0.5)보다는 높아야** 합니다. 0.5는 그냥 찍는 동전 던지기 — 그보다 조금이라도 나으면 모을 자격이 생겨요.

03 0.5보다 나으면 — 0으로 수렴

오류율 0.3짜리 모델도 25개를 모으면 다수결 오류율이 **0.017**까지 떨어집니다. 머릿수가 늘수록 0에 가까워져요 — 약한 모델도 모이면 강해집니다.

04 0.5보다 못하면 — 더 나빠진다

오류율 0.6짜리를 모으면? 25개 다수결 오류율이 오히려 **0.85**로 치솟습니다. 다 같이 틀리는 쪽으로 쏠리는 거예요 — 약한 모델은 모을수록 더 나빠집니다.

앙상블의 부품으로 — 결정 트리가 제격인 이유

수백 개를 모으려면 부품 하나가 **빠르고**, **까다롭지 않고**, **서로 달라지기 쉬워야** 합니다. 결정 트리가 정확히 그래요.

빠름

수백 그루도 금방

트리는 학습 비용이 낮아 데이터가 커도 빠르게 세워집니다. 한 개라면 의미 없지만, **수백 개를 길러야 하는** 앙상블에선 이 속도가 결정적이에요 — 게다가 그루마다 병렬로 키울 수 있죠.

비모수

데이터 분포를 안 따진다

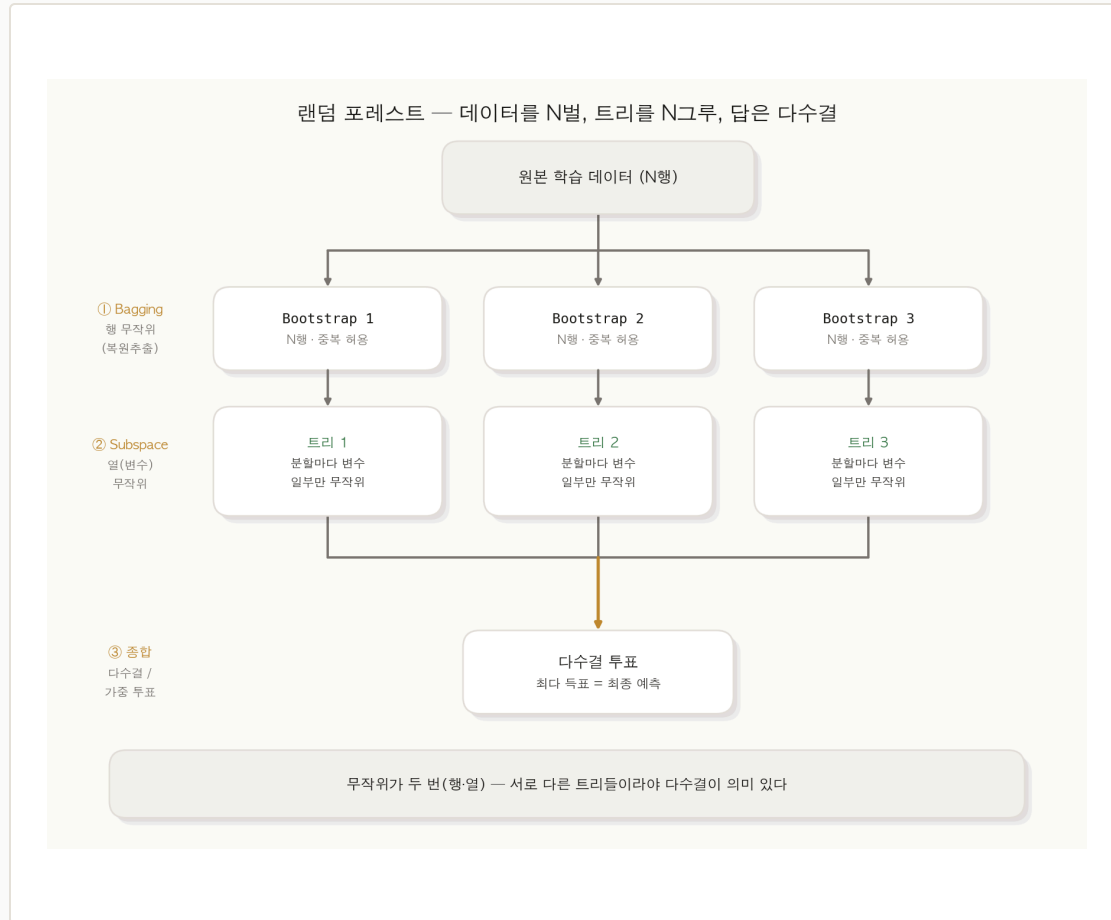
트리는 **Nonparametric** — "데이터가 정규분포일 것" 같은 전제가 필요 없습니다. 전처리 부담이 적어 아무 표 데이터에나 바로 엮을 수 있어요. 까다롭지 않은 부품이라 대량 생산에 적합합니다.

다양성

약점이 곧 강점이 된다

트리는 데이터가 조금만 달라도 **전혀 다른 나무**가 됩니다 — 한 그루로 쓰면 번덕이지만, 여럿을 모을 땐 **서로 달라지기 쉽다**는 뜻이에요. 앙상블에 필요한 이 다양성을, 트리는 자연히 갖고 있는 셈이죠.

복제하고, 따로 키우고, 투표한다 — 세 단계



원본 한 벌에서 Bootstrap 여러 벌을 복제(①), 각 벌로 변수를 일부만 보며 트리를 따로 키우고(②), 마지막에 다수결로 종합한다(③).

01 ① 복제 — 데이터를 N벌로

원본 학습 데이터에서 **복원추출**로 여러 벌의 데이터를 만듭니다. 벌마다 구성이 조금씩 달라요 — 이게 행(관측치)의 무작위, **Bagging**입니다.

02 ② 따로 키우기 — 트리를 한 그루씩

각 벌로 트리를 한 그루씩 세웁니다. 이때 분기마다 **변수도 일부만 무작위로** 봐요 — 열(변수)의 무작위, **Random subspace**죠. 나무들이 서로 더 달라집니다.

03 ③ 투표 — 답을 종합한다

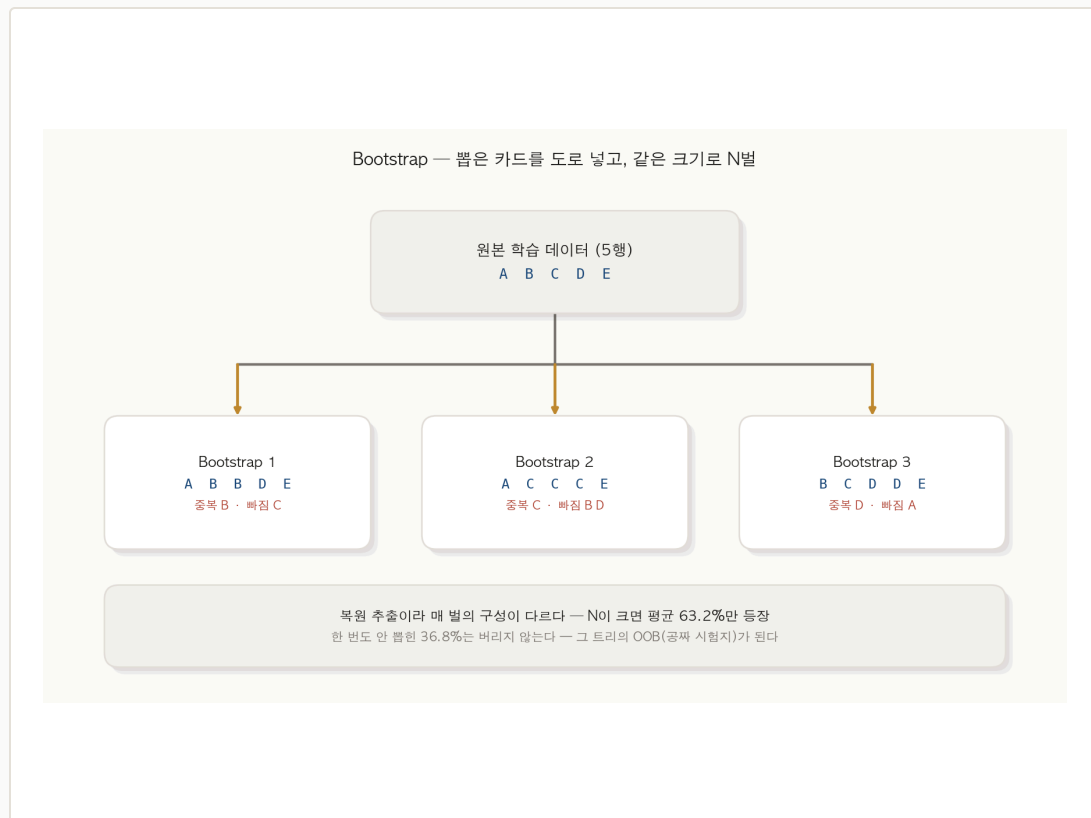
새 데이터가 오면 모든 트리가 각자 예측하고, 그 결과를 **다수결**(또는 확률 평균)로 모읍니다. 흔들리는 한 그루들이 모여 **안정된 하나의 답**이 돼요.

04 두 무작위가 핵심

행도 다르고 열도 다른 — **두 번의 무작위**가 트리들을 서로 닮지 않게 만듭니다. §4.3의 "독립" 조건을 일부러 만들어 주는 장치예요. 이제 둘을 뜯어봅니다.

Bagging의 절반 — 행(관측치)을 다시 뽑는다

Bagging = Bootstrap + Aggregating. 먼저 **Bootstrap(부트스트랩)** — 원본 표에서 **행(관측치)**을 복원추출해 여러 벌의 데이터를 만듭니다. 뽑은 행을 도로 넣고 다시 뽑으니, 각 벌은 원본과 **같은 크기**지만 구성이 저마다 달라요. (합치는 Aggregating은 §5.3)



01 복원추출이라 중복·누락이 생긴다

뽑은 행(학생 한 명)을 **도로 넣고** 다시 뽑으니, 한 벌 안에 같은 행이 두 번 들기도 하고 한 번도 안 들기도 합니다. 그래서 벌마다 성격이 달라져요.

02 크기는 원본과 똑같이

각 부트스트랩 벌은 원본과 **같은 N행**입니다. 중복으로 채우니, 실제 등장하는 서로 다른 행은 그보다 적어요.

03 평균 63.2%만 등장한다

n 이 크면 한 벌에 원본의 약 **63.2%**만 나타납니다 — 한 행이 안 뽑힐 확률이 $(1 - \frac{1}{n})^n \rightarrow \frac{1}{e}$ 라서요.

04 남은 36.8%는 버리지 않는다

한 번도 안 뽑힌 **36.8%**는 그 트리가 **본 적 없는** 데이터입니다. 이게 §5.6의 OOB — 검증용 데이터로 다시 쓰입니다.

Bagging의 나머지 — 트리들의 답을 종합한다(Aggregating)



따로 키운 트리들이 각자 예측하면, 그 답을 다수결(또는 평균)로 합쳐 하나의 답을 낸다 — 합치는 방식에 따라 결과가 뒤집히기도 한다.

01 각자 예측을 모은다

새 데이터가 오면 트리마다 **각자 답**을 냅니다. 이 여러 답을 **하나로 합치**는 단계가 Aggregating(종합) — Bagging의 "Agg"예요.

02 기본은 다수결 · 평균

분류는 **표를 세서** 가장 많은 라벨로 정합니다(트리 5그루가 1·1·1·0·0 → 다수결 **1**). 회귀라면 다섯 예측값의 **평균**을 내고요.

03 가중 투표 · 확률 평균도 있다

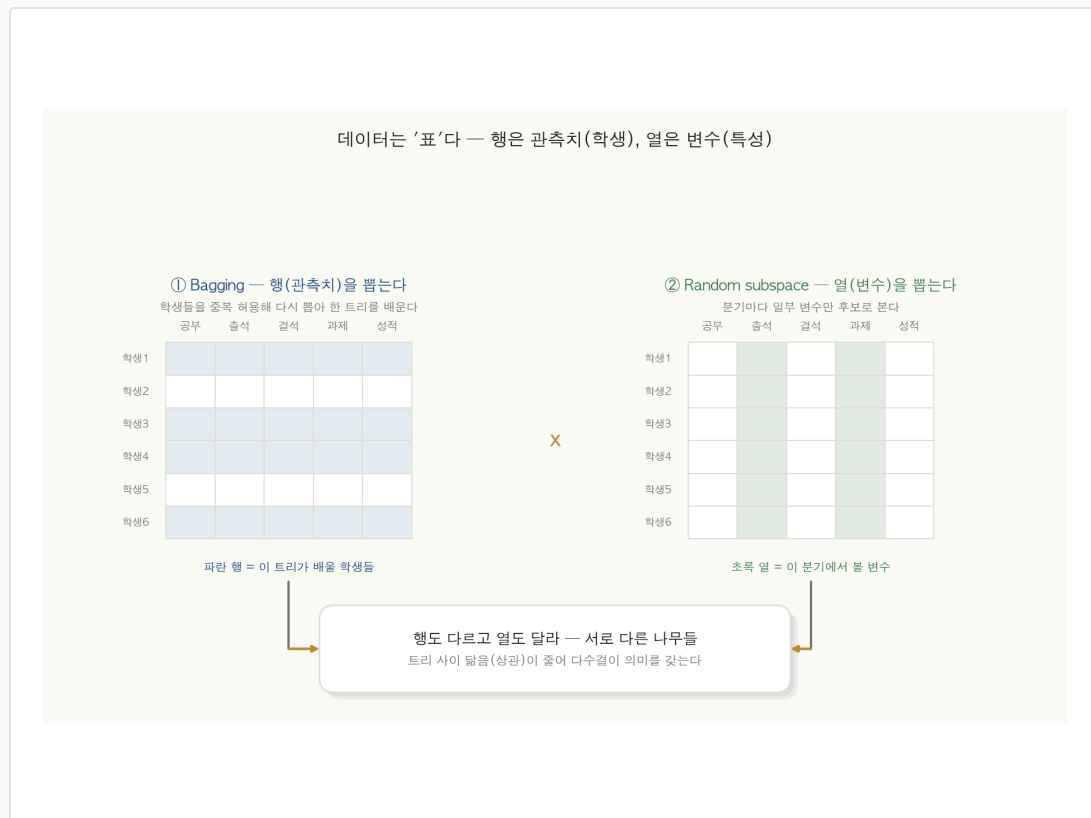
정확도가 높은 트리의 표를 더 무겁게 주거나(정확도 가중 투표), 표 대신 각 트리의 **예측 확률을 평균**낼 수도 있어요 — "얼마나 확신하는지"까지 반영하죠.

04 종합 방식이 답을 바꾸기도

위 그림에선 단순 다수결도 정확도 가중도 **합격(1)**인데, 확률 평균만 **불합격(0)** — 셋은 0.55쯤의 미지근한 찬성, 둘은 0.1 이하의 강한 반대라 확신의 크기를 들으면 뒤집혀요.

변수도 무작위로 — 두 번째 무작위

Random subspace는 트리를 키울 때 **분기마다 전체 변수 중 일부만 무작위로** 후보에 올리는 기법입니다. Bagging이 **행(관측치)**을 뽑았다면, 이건 **열(변수)**을 무작위로 가리는 거예요. 표에서 행은 **학생(관측치)**, 열은 **공부·출석 같은 변수** — 두 무작위가 합쳐져 트리들을 더 멀어지게 합니다.



01 다양성 ↑ — 트리가 더 달라진다

각 트리가 **다른 변수 묶음**으로 배우니, 같은 데이터에서도 전혀 다른 나무가 자랍니다. 트리 간 상관이 더 떨어져요.

02 과적합 ↓ — 강한 변수의 독점을 막는다

한 변수가 너무 세면 모든 트리가 거기에 매달립니다. 변수를 가려두면 **약한 변수도 기회**를 얻어, 숲이 한쪽으로 쏠리지 않아요.

03 계산 ↓ — 후보가 적으니 빠르다

분기마다 전체 변수가 아닌 **일부만** 따지니 한 그루 세우는 속도가 빨라집니다. 수백 그루를 길러야 하는 입장에선 큰 절약이죠.

04 두 무작위의 합 — 행 × 열

행(Bagging) × 열(subspace) 두 무작위가 곱해져, §4.3이 요구한 "독립"에 가까운 다양성을 만듭니다. 이게 '랜덤' 포레스트의 두 '랜덤'이에요.

트리는 외우는데, 숲은 왜 안 외울까

개별 트리는 깊게 자라 과적합됩니다. 그런데 그런 나무를 수백 그루 모은 숲은 — 오히려 과적합이 **찾아들어요**. 어떻게 이런 일이 가능할까요?

숲의 최악 성능엔 상한이 있습니다. **일반화 오차**는 모델이 새 데이터에서 보일 수 있는 **최악의 오차**예요. 랜덤 포레스트의 일반화 오차에는 수학적 **상한(upper bound)**이 있는데, 브레이먼은 그 상한을 $PE^* \leq \frac{\bar{\rho}(1-s^2)}{s^2}$ 로 보였습니다 — 딱 두 숫자로 정해지죠.

두 숫자 — **상관 $\bar{\rho}$ 와 정확도 s** . $\bar{\rho}$ 는 트리들 사이의 **평균 상관**(서로 얼마나 닮았나), s 는 개별 트리의 **평균 정확도**(한 그루가 얼마나 잘 맞히나)입니다. 식을 보면 $\bar{\rho}$ 가 작을수록, s 가 클수록 상한이 낮아져요 — 외울 건 이 방향뿐입니다.

두 무작위가 정확히 $\bar{\rho}$ 를 낮춥니다. Bagging과 Random subspace는 트리들을 **서로 닮지 않게** 만드는 장치였죠 — 즉 **상관 $\bar{\rho}$ 를 끌어내립니다**. 개별 트리의 정확도 s 는 크게 해치지 않으면서요. 그래서 상한이 내려가고, 새 데이터 성능이 좋아집니다.

큰 수의 법칙이 이를 보장합니다. 트리 수가 충분히 많아지면, 숲의 오차는 더 출렁이지 않고 **어떤 한계값에 수렴**합니다. 나무를 더 심어도 **나빠지진 않아요** — 이것이 "포레스트는 과적합하지 않는다"는 말의 뜻입니다. 이제 그 다양성이 **공짜 검증과 변수 중요도**라는 덤까지 준다는 걸 봅시다.

안 뽑힌 36.8%가 공짜 시험지가 된다 — OOB



각 트리는 자기 bootstrap에 안 뽑힌 약 36.8%(OOB)를 본 적이 없다 — 바로 그 데이터로 그 트리를 채점하면, 별도 검증 분할 없이 테스트 성능을 추정할 수 있다.

01 Bagging이 남긴 데이터

복원추출 때문에 각 트리가 **못 본 36.8%**가 생긴다고 했죠 (§5.2). 트리 입장에선 이게 **처음 보는 데이터** — 검증에 쓰기 좋습니다.

02 Out-Of-Bag = 가방 밖

한 트리의 bootstrap '가방'에 안 담긴 데이터라 **Out-Of-Bag**입니다. 그 트리는 OOB를 학습에 쓴 적이 없으니, OOB로 채점하면 **공정한 시험**이 돼요.

03 별도 분할이 필요 없다

보통은 train/test를 따로 나눠 검증하지만, OOB가 있으면 **데이터를 쪼개지 않고도** 성능을 추정합니다. 데이터가 부족할 때 특히 쓸모 있죠.

04 거의 공짜다

학습 과정에서 자연스럽게 생기는 부산물이라 **추가 계산이 거의 없어요**. 숲을 다 키우면 OOB 점수가 함께 나와, 따로 나눈 test 점수와 거의 일치합니다.

섞어보면 안다 — 어떤 변수가 중요한가

랜덤 포레스트는 로지스틱 회귀처럼 깔끔한 계수를 안 줘요. 대신 **간접적으로** 중요도를 잽니다 — 한 변수의 값을 **마구 섞은 뒤** OOB Error가 얼마나 나빠지는지를 보는 **permutation** 방식이에요.

01 1단계 — 섞기 전 OOB Error

먼저 원래 데이터로 OOB Error를 잽니다. 이게 **기준점**이에요 — 그림의 회색 막대 (0.12)죠.

02 2단계 — 한 변수만 섞기

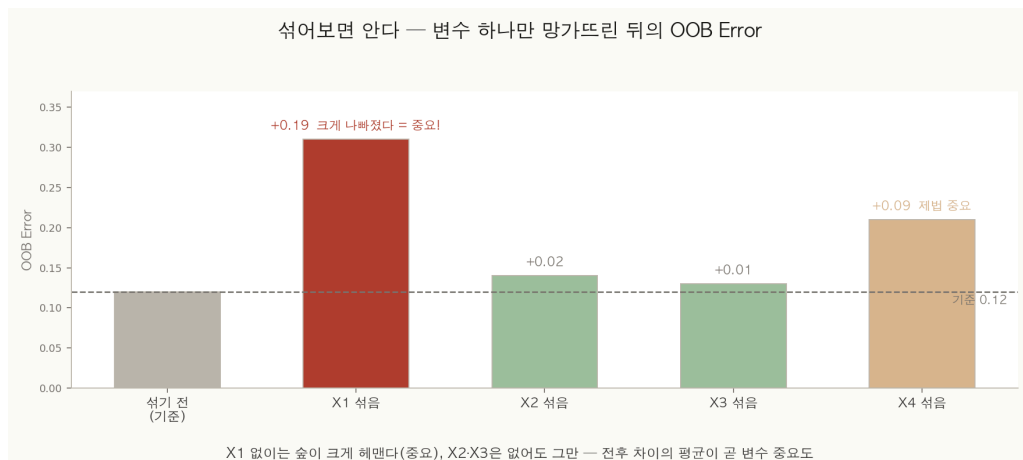
특정 변수의 값만 **헝끼리 마구 뒤섞어** 그 변수를 의미 없게 만든 뒤, 다시 OOB Error를 잽니다. 다른 변수는 그대로 뒹요.

03 차이가 크면 = 중요

섞었더니 OOB가 **크게 나빠졌다면** (X_1 : +0.19) 숲이 그 변수에 크게 기대고 있었다는 뜻 — 중요한 변수입니다. 거의 그대로면 (X_2) 없어도 그만이죠.

04 전후 차이를 평균한다

섞은 후 오차 e_i 에서 섞기 전 오차 r_i 를 뺀 $d_i = e_i - r_i$ 를 모든 트리에서 평균 낸 값이 i 번째 변수의 중요도예요. 고차원 데이터의 **변수 선택**에 강력합니다.



결정 트리 한 그루 vs 랜덤 포레스트

같은 트리에서 출발하지만, **한 그루냐 수백 그루냐**가 모든 걸 바꿉니다. 숲의 약점도 정직하게 짚어요.

투명하지만 변덕스럽다

결정 트리 한 그루

분산이 크다

데이터가 조금만 달라져도 **전혀 다른 나무**가 됩니다. 노이즈에 약하고 깊게 키우면 과적합해요 — Low Bias, High Variance.

완벽하게 해석된다

"소득 > 5천 & 나이 < 40 → 승인"처럼 **규칙 한 줄**로 예측 이유를 따라 읽을 수 있습니다. 그림 한 장이면 끝이죠.

빠르고 가볍다

한 그루뿐이라 학습·예측이 빠릅니다. 작고 단순한 문제, 또는 **설명이 최우선**인 곳에 어울려요.

안정적이지만 흐려진다

랜덤 포레스트 (숲)

분산이 작다

수백 그루의 다수결이라 **좀처럼 흔들리지 않습니다**. 노이즈·이상치에 강하고, 트리가 많아져도 과적합하지 않아요 (§5.5).

해석이 흐려진다

수백 그루의 합의라 **규칙 한 줄로 설명하기 어렵습니다**. 대신 **변수 중요도** (§5.7)로 "무엇이 중요했나"는 답할 수 있어요 — 한 그루만큼 투명하진 않습니다.

느리고 무겁다

수백 그루를 길러야 하니 한 그루보다 **느리고 메모리도 더 씁니다**. 그 대신 정확도와 안정성을 얻습니다 — 대개 그만한 값을 합니다.

정리 — 질문의 나무에서 다수결의 숲으로

오늘 한 일

- ▶ **결정 트리** — 질문을 거듭해 쪼개는 규칙 기반 분류기, 루트·노드·잎·깊이
- ▶ **분할** — 모든 j·s를 시험해 불순도(지니·엔트로피·정보 이득)를 가장 줄이는 질문
- ▶ **가지치기** — max_depth·min_samples로 한 그루의 과적합을 다스린다
- ▶ **앙상블** — 독립이고 0.5보다 나은 모델을 모으면 강해진다
- ▶ **랜덤 포레스트** — 두 무작위(행·열)로 다양성, OOB·변수 중요도가 덩

직선 대신 질문으로 가르고(트리), 어디서 쪼갤지는 불순도가 정하며(지니·정보 이득), 너무 깊어지면 가지를 친다.
한 그루는 투명하지만 흔들린다 — 그래서 두 번의 무작위로 서로 다르게 기른 수백 그루를 다수결로 묶으면(랜덤 포레스트) 상관이 낮아져 안정적이고, OOB·변수 중요도까지 공짜로 얻는다. 이 앙상블의 발상이 부스팅으로 이어진다.

더 나아가려면

- ▶ **불순도 두 자 비교** — 지니와 엔트로피로 같은 데이터를 갈라 결과가 비슷한지 확인
- ▶ **깊이와 가지치기** — 깊이를 바꿔 가며 train·test 점수가 갈리는 지점(과적합) 관찰
- ▶ **행·열 두 무작위** — Bagging(행)·Random subspace(열)이 트리 간 상관을 어떻게 낮추는지
- ▶ **OOB와 변수 중요도** — 안 뽑힌 36.8%로 공짜 검증, 변수 섞기로 영향력 측정
- ▶ **부스팅으로** — 같은 앙상블 발상의 다른 길, GBDT·XGBoost·LightGBM